

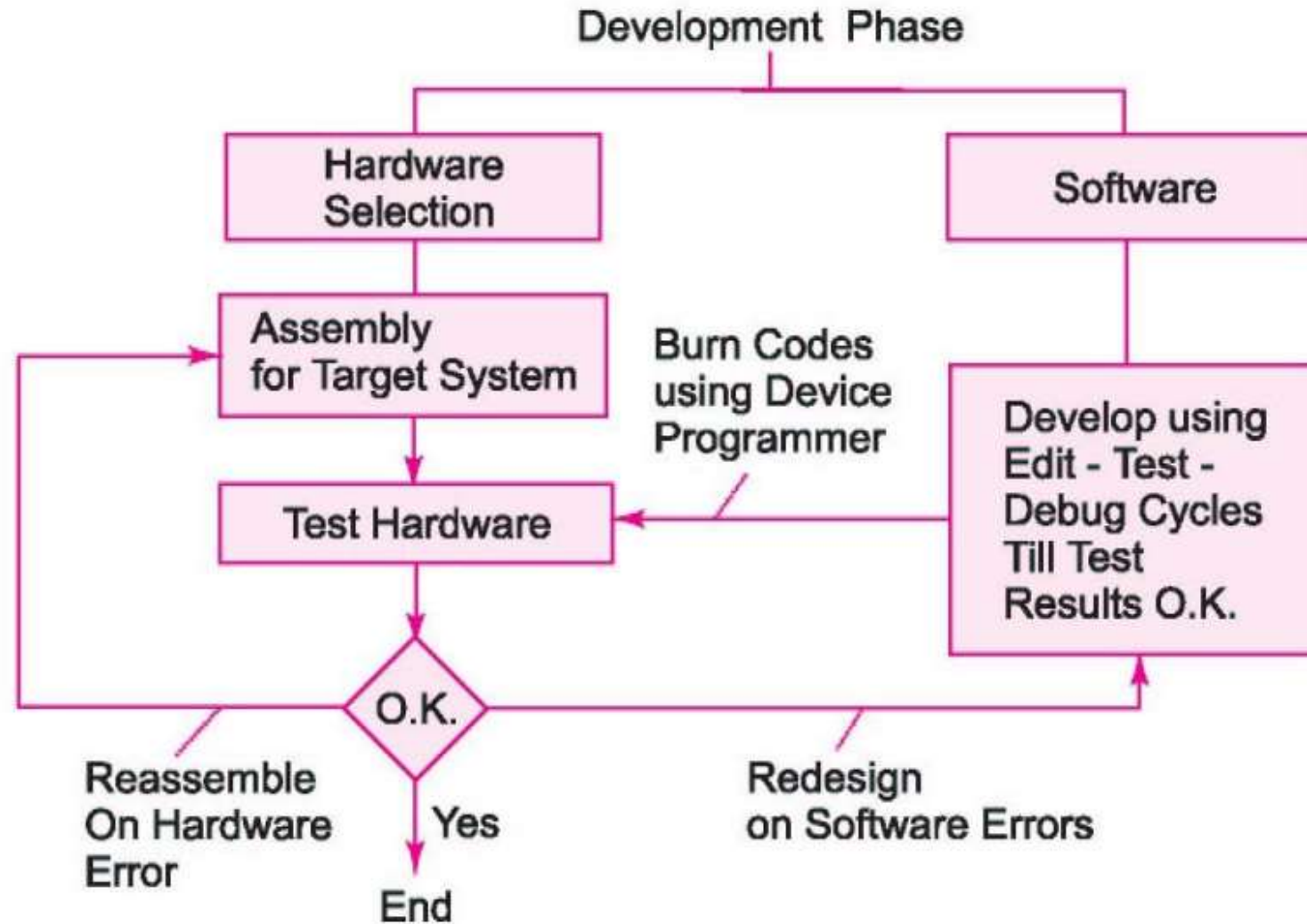
MODULE-4

Programming tools

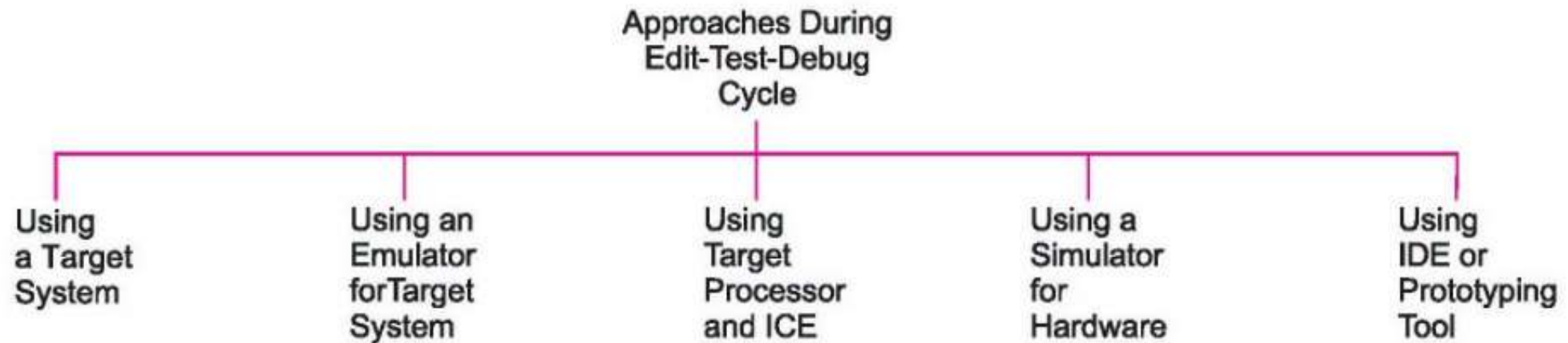
Evolution of embedded programming tools,
Modelling programs, Code optimization, Logic
analyzers, Programming environment

BUILDING PROCESS FOR EMBEDDED SYSTEMS

BUILDING PROCESS FOR EMBEDDED SYSTEMS



BUILDING PROCESS FOR EMBEDDED SYSTEMS



Edit-Test-Debug Cycle implementation phase of the development process

BUILDING PROCESS FOR EMBEDDED SYSTEMS

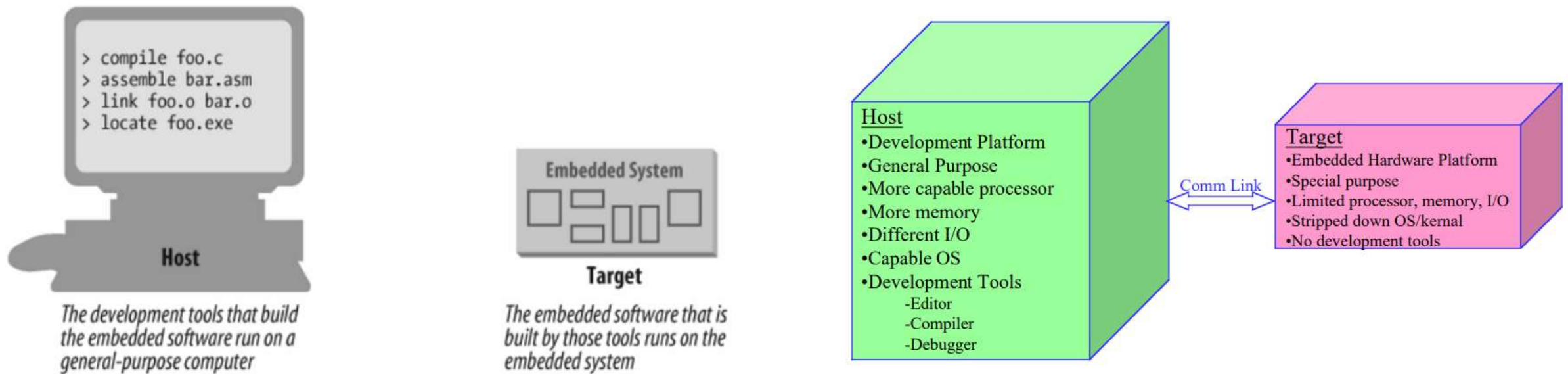
- ❑ Embedded systems programming is not substantially different from the programming you've done before.
- ❑ The only thing that has really changed is that you need to have an **understanding of the target hardware platform**. Furthermore, each target hardware platform is **unique**.
- ❑ Unfortunately, this uniqueness among hardware platforms leads to a lot of **additional software complexity**, and it's also the reason you'll need to be more aware of the **software build process** than ever before.

BUILDING PROCESS FOR EMBEDDED SYSTEMS

- ❑ When build tools run on the same system as the program they produce, they can make a lot of assumptions about the system.
- ❑ This is typically not the case in embedded software development, where the build tools run on a host computer that differs from the target hardware platform.
- ❑ Embedded software development tools, can rarely make assumptions about the target platform.
- ❑ Instead, the user must provide some knowledge of the system to the tools by giving them more explicit instructions.

BUILDING PROCESS FOR EMBEDDED SYSTEMS

- ❑ **Software Development is performed on a Host computer**
 - **Compiler, Assembler, Linker, Locator, Debugger**
 - **Produces executable binary image that will run on Target Embedded System**



BUILDING PROCESS FOR EMBEDDED SYSTEMS

❑ Software Tools

1. Software Development Kit (SDK)
2. Source-code Engineering Software
3. RTOS
4. Integrated Development Environment (IDE)
5. Emulator
6. Editor
7. Interpreter
8. Compiler
9. Assembler
10. Cross Assembler
11. Locator
12. Testing and debugging tools

PROCESS FOR DEVELOPING EMBEDDED SOFTWARE

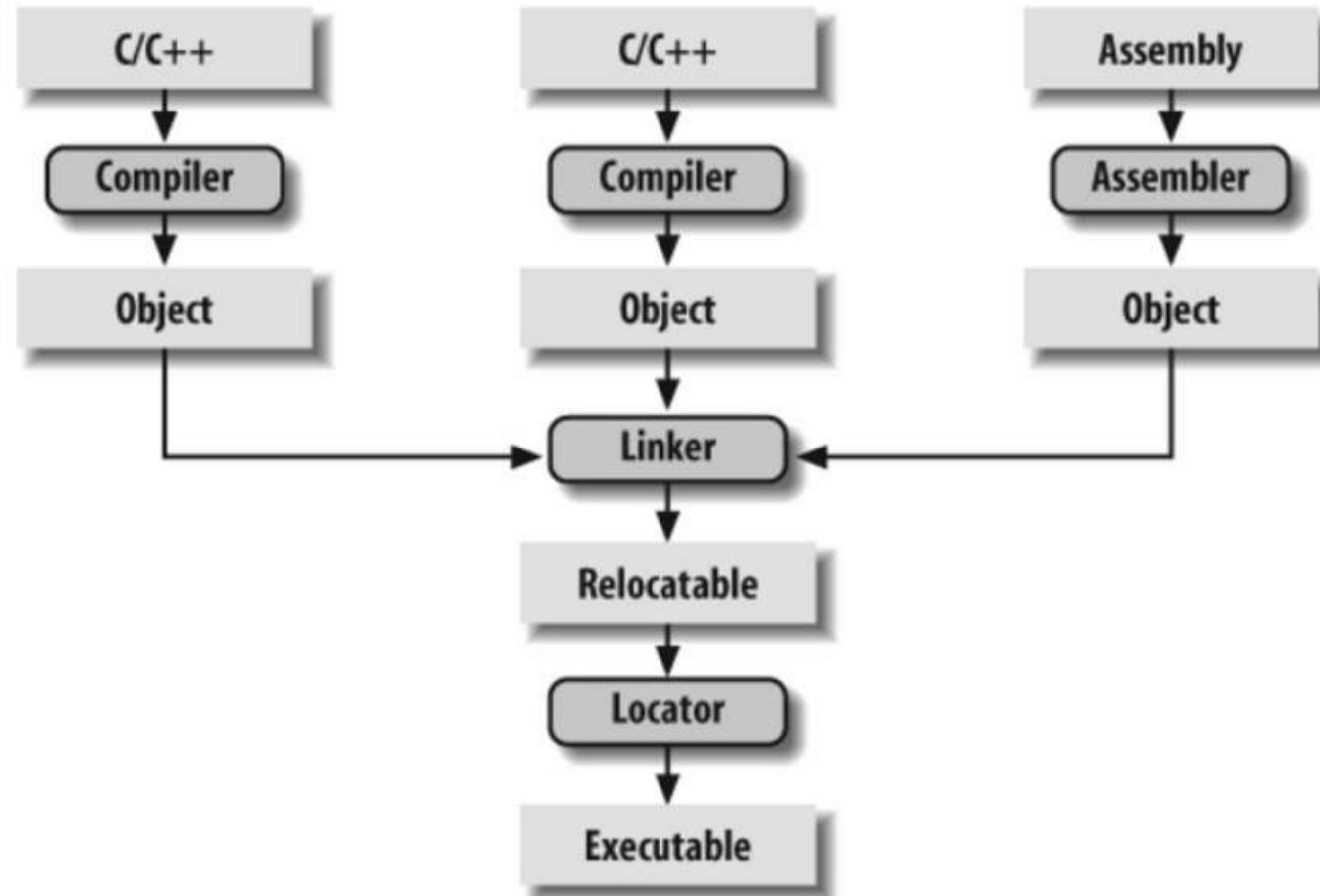
❑ To develop software for a **General Purpose Computer**

- Create source file
- Type in C code
- Build: compile and link
- Execute: load and run

❑ To develop software for an **embedded system**

- Create source file (on Host)
- Type in C code (on Host)
- Compile/Assemble: translate into machine code (on Host)
- Link: combine all object files and libraries, resolve all symbols (on Host)
- Locate: assign memory addresses to code and data (on Host)
- Download: copy executable image into Target processor memory
- Execute: reset Target processor

PROCESS FOR DEVELOPING EMBEDDED SOFTWARE



The Embedded Software Development Process

PROCESS FOR DEVELOPING EMBEDDED SOFTWARE

SOURCE CODE INTO EXECUTABLE BINARY IMAGE

Each of the source files must be compiled or assembled into an object file



All of the object files that result from the first step must be linked together to produce a single object file, called the re-locatable program.



Physical memory addresses must be assigned to the relative offsets within the re-locatable program in a process called relocation.

PREPROCESSING

- ❑ Pre-processing is **not a part of the compiler**, but is a separate step in the compilation process
- ❑ Pre-processing is a process of **running a set of instructions** provided by the programmer explicitly as code before the program compiles.
- ❑ The pre-processor provides the ability for the inclusion of **header files, macro expansions, conditional compilation, and line control**.
- ❑ Pre-processor instructions are called pre-processor directives, and they all start with a hash symbol (#). Few examples: **"#include"**, **"#define"**, **"#line"**, and many more.

COMPILING

- ❑ **Compiler** translate programs written in some human-readable language into an equivalent set of op-codes (or machine language) for a particular processor.
- ❑ Compiler performs conversion of **Source Code --> Object file**
- ❑ Object file is binary file that contains **set of machine-language instructions** (opcodes) and data resulting from language translation Process
- ❑ Each processor has its own unique machine language, so you need to choose a **compiler that produces programs for your specific target processor.**

CROSS COMPILING

- ❑ In the embedded systems case, compiler almost always **runs on the host computer.**
- ❑ A **Native-compiler** runs on a computer platform and produces code for that same computer platform
- ❑ A **Cross-compiler** runs on one computer platform and produces code for another computer/target platform
- ❑ The use of a cross-compiler is one of the **defining features of embedded software development.**

COMPILER - EXAMPLE

- ❑ The GNU C compiler (gcc) and assembler (as) can be configured as either native compilers or cross-compilers.
- ❑ These tools support an impressive set of host-target combinations.
- ❑ The gcc compiler will run on all common PC and Mac operating systems.
- ❑ The target processor support is extensive, including AVR, Intel x86, MIPS, PowerPC, ARM, and SPARC.

COMPILER – OBJECT FILE FORMAT

- ❑ Regardless of the input language (C, C++, assembly, or any other), the output of the cross-compiler will be an object file.
- ❑ Although parts of this file contain executable code, the object file cannot be executed directly.
- ❑ The contents of an object file can be thought of as a very large, flexible data structure.
- ❑ The structure of the file is often defined by a standard format such as the Common Object File Format (COFF) or Executable and Linkable Format (ELF).

COMPILER – OBJECT FILE

- ❑ Most object files **begin with a header** that describes the sections that follow.
- ❑ Each of these sections **contains one or more blocks of code or data** that originated within the source file you created.
- ❑ However, the compiler has **regrouped these blocks** into related sections.
- ❑ For example, in gcc
 - **text** - all of the code blocks are collected into this section,
 - **data** - initialized global variables (and their initial values) into this section
 - **bss** - uninitialized global variables into this section.

COMPILER – SYMBOL TABLE

- ❑ There is also usually a symbol table somewhere in the object file that **contains the names and locations of all the variables and functions** referenced within the source file.
- ❑ Parts of this table may be **incomplete**, however, because not all of the variables and functions are always defined in the same file.
- ❑ These are the symbols that **refer to variables and functions defined in other source files**.
- ❑ And it is up to the **linker to resolve** such unresolved references.

LINKER

- ❑ All of the object files, resulting from the compilation in step one, must be **combined**.
- ❑ The object files themselves are individually **incomplete**, some of the internal variable and function references not yet been resolved.
- ❑ The job of the linker is to combine these object files and, in the process, to resolve all of the unresolved symbols
- ❑ By merging the text, data, and bss sections of the input object files, the linker creates a new object file that contains all of the code and data from the input object files.

LINKER

- ❑ When the linker is finished executing, all of the machine language code from all input object files will be in **text section** of the new file.
- ❑ And all of the initialized and uninitialized variables will reside in the new **data and bss** sections, respectively.
- ❑ After merging all of the code and data sections and resolving all of the symbol references, the linker produces an object file that is a special “**relocatable**” copy of the program.
- ❑ In other words, the program is complete except for one thing: **no memory addresses have yet been assigned** to the code and data sections within.

LOCATOR

- ❑ The tool that performs the conversion from relocatable program to executable binary image is called a locator.
- ❑ You have to do most of the work in this step yourself, by providing information about the memory on the target board as input to the locator.
- ❑ The locator uses this information to assign physical memory addresses to each of the code and data sections within the relocatable program.
- ❑ It then produces an output file that contains a binary memory image that can be loaded into the target.

LOCATOR

- ❑ The output of this final step of the build process is a binary image containing **physical addresses for the specific embedded system**.
- ❑ This executable binary image can be **downloaded to the embedded system** or programmed into a memory chip.
- ❑ There is a separate development tool, called a **locator**, to assign addresses. However, in the of GNU tools, **this feature is built into the linker (ld)**.
- ❑ The memory information required by the GNU linker can be passed to it in the form of a **linker script**.

LOCATOR - EXAMPLE

example of a linker script

```
ENTRY (main)

MEMORY
{
    ram : ORIGIN = 0x00400000, LENGTH = 64M
    rom : ORIGIN = 0x60000000, LENGTH = 16M
}

SECTIONS
{
    data :                /* Initialized data. */
    {
        _DataStart = . ;
        *(.data)
        _DataEnd = . ;
    } >ram

    bss :                  /* Uninitialized data. */
    {
        _BssStart = . ;
        *(.bss)
        _BssEnd = . ;
    } >ram

    text :                 /* The actual instructions. */
    {
        *(.text)
    } >ram
}
```

```
AREA    ARMex, CODE, READONLY

        ENTRY              ; Name this block of code ARMex
start   ; Mark first instruction to execute

        MOV    r0, #10     ; Set up parameters
        MOV    r1, #3
        ADD    r0, r0, r1  ; r0 = r0 + r1

stop    ;

        MOV    r0, #0x18   ; angel_SWIreason_ReportException
        LDR    r1, =0x20026 ; ADP_Stopped_ApplicationExit
        SVC    #0x123456   ; ARM semihosting (formerly SWI)
        END              ; Mark end of file
```


LOCATOR - EXAMPLE

- ❑ The first executable instruction is **ENTRY** command, which appears on the first line and the **entry point** is the function **main**.
- ❑ This script informs the GNU linker's built-in locator about the memory on the target board, which contains **64 MB of RAM** and **16 MB of flash ROM**.
- ❑ The **linker script file** instructs the GNU linker to locate the **data, bss, and text sections** in RAM starting at address **0x00400000**.
- ❑ Names in the **linker command file** that begin with an **underscore** (e.g., **_DataStart**) can be referenced similarly to ordinary variables from source code.

LOCATOR - EXAMPLE

- ❑ The linker will use these symbols to resolve references in the input object files.
- ❑ So, for example, there might be a part of the embedded software (usually within the startup code) that copies the initial values of the initialized variables from ROM to the data section in RAM.
- ❑ The start and stop addresses for this operation can be established symbolically by referring to the addresses as `_DataStart` and `_DataEnd`.

BUILD PROCEDURE FOR THE ARCOM VIPER-LITE DEVELOPMENT BOARD

- ❑ In this section, we show an example build procedure for the Arcom VIPER-Lite development board.
- ❑ If another hardware platform is used, a similar process should be followed using the tools and conventions that accompany that hardware.
- ❑ Once the tools are installed, the commands covered in the following sections are entered into a command shell.
- ❑ For Windows users, the command shell is a Cygwin bash shell (Cygwin is a Unix environment for Windows); for Linux users, it is a regular command shell.

BUILD PROCEDURE FOR THE ARCOM VIPER-LITE DEVELOPMENT BOARD

COMPILE

- ❑ First we look at the individual commands in order to manually perform the three separate tasks (compiling, linking, and locating)
- ❑ Later, we will learn how to automate the build procedure with makefiles.
- ❑ The Blinking LED example consists of two source modules: *led.c* and *blink.c*.
- ❑ The first step in the build process is to compile these two files.

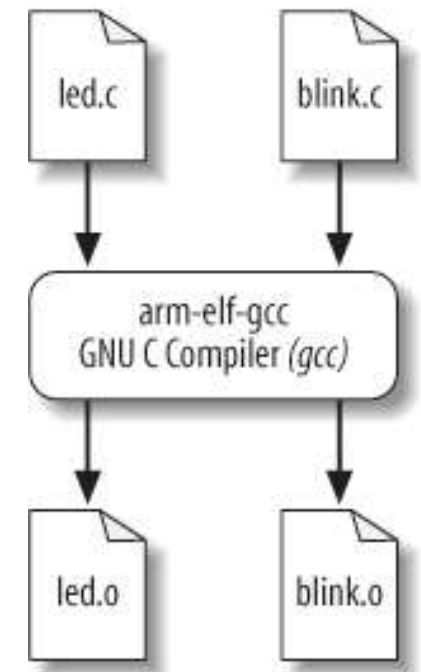
BUILD PROCEDURE FOR THE ARCOM VIPER-LITE DEVELOPMENT BOARD

COMPILE

- ❑ The basic structure for the gcc compiler command is:
- ❑ The command-line options we'll need are:
 - g** To generate debugging info in default format
 - c** To compile and assemble but not link
 - Wall** To enable most warning messages
 - I../include** To look in the directory include for header files
- ❑ Here are the actual commands for compiling:

```
# arm-elf-gcc -g -c -Wall -I../include led.c
# arm-elf-gcc -g -c -Wall -I../include blink.c
```

```
arm-elf-gcc [  
    options  
]  
    file  
...
```



BUILD PROCEDURE FOR THE ARCOM VIPER-LITE DEVELOPMENT BOARD

COMPILE

- ❑ We broke up the compilation step into **two separate commands**, but you can compile the two files with one command.
- ❑ To use a single command, just put **both of the source files after the options**. If you wanted different options for one of the source files, you would need to compile it separately as just shown.
- ❑ The result of each of these commands is the creation of an object file that has the **same prefix as the .c file, and the extension .o**.
- ❑ So if all goes well, there will now be two additional files—**led.o and blink.o**—in the working directory.

BUILD PROCEDURE FOR THE ARCOM VIPER-LITE DEVELOPMENT BOARD

LINK AND LOCATE

- ❑ We now have the two object files—led.o and blink.o—that we need in order to perform the **second step in the build process**.
- ❑ As we discussed earlier, the **GNU linker** performs the **linking and locating** of the object files.
- ❑ For the third step, **locating**, there is a linker script file named **viperlite.ld** that we input to ld in order to establish the location of each section in the Arcom board's memory.

BUILD PROCEDURE FOR THE ARCOM VIPER-LITE DEVELOPMENT BOARD

LINK AND LOCATE

❑ The structure for the linker and locator ld command is:

```
arm-elf-ld [
                options
            ]
            file
            ...
```

❑ The command-line options we'll need for this step are:

- | | |
|------------------------|--|
| -Map blink.map | To generate a map file and use the given filename |
| -T viperlite.ld | To read the linker script |
| -N | To set the text and data sections to be readable and writable |
| -o blink.exe | To set the output filename (if this option is not included, ld will use the default output filename a.out) |

❑ The actual command for linking and locating is:

```
# arm-elf-ld -Map blink.map -T viperlite.ld -N -o blink.exe led.o blink.o
```


BUILD PROCEDURE FOR THE ARCOM VIPER-LITE DEVELOPMENT BOARD

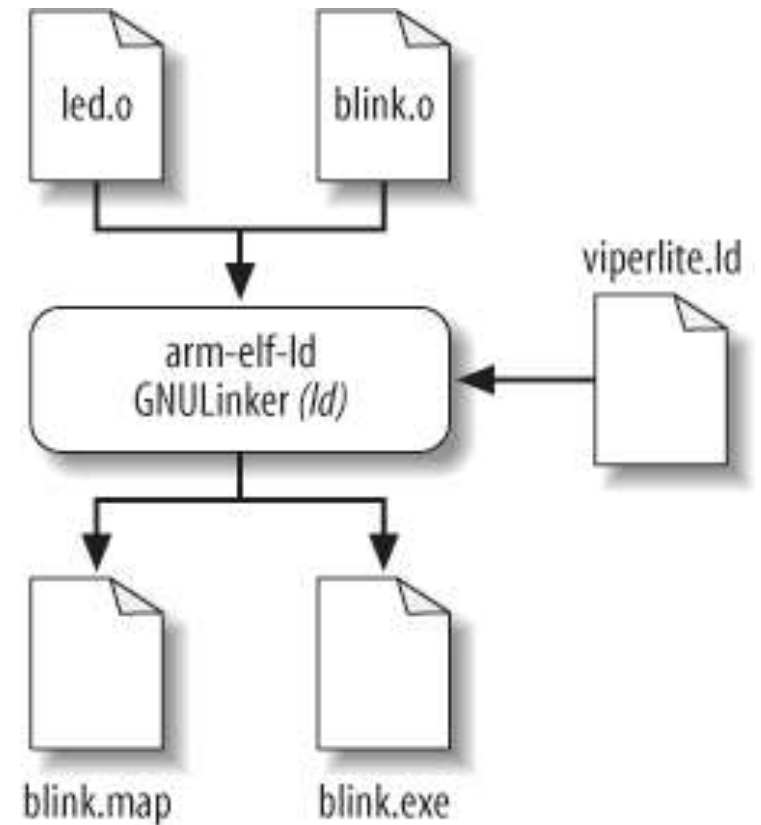
LINK AND LOCATE

- ❑ The order of the object files determines their placement in memory. Because we are not linking in any startup code, the order of the object files is irrelevant.
- ❑ If startup code were included, you would want that object file to be located at the proper address.
- ❑ The linker script file can be used to specify where you want the startup routine (and other code) to reside in memory.
- ❑ Furthermore, you can also use the linker script file to specify exact addresses for code or data, should you find it necessary to do so.

BUILD PROCEDURE FOR THE ARCOM VIPER-LITE DEVELOPMENT BOARD

LINK AND LOCATE

- ❑ As you can see in this command, the **two object files—led.o and blink.o**—are the last arguments on the command line for linking.
- ❑ The linker script file, **viperlite.ld**, is also passed in for locating the data and code in the Arcom board's memory.
- ❑ The result of this command is the creation of **two files—blink.map and blink.exe**—in the working directory.



BUILD PROCEDURE FOR THE ARCOM VIPER-LITE DEVELOPMENT BOARD

LINK AND LOCATE

- ❑ The .map file gives a complete listing of all code and data addresses for the final software image.
- ❑ It provides information similar to the contents of the linker script described earlier.
- ❑ However, these are results rather than instructions and therefore include the actual lengths of the sections and the names and locations of the public symbols found in the relocatable program.

LINKER MAP FILES

- ❑ The linker map file provides valuable information that can help you understand and optimize memory.
- ❑ The map file is a symbol table for the whole program.
- ❑ The most straightforward pieces of information in the map file are the actual memory regions, with location, size and access rights granted to those regions

Memory Configuration

Name	Origin	Length	Attributes
FLASH	0x00000000000001000	0x000000000000ff000	xr
RAM	0x0000000020000008	0x0000000000003fff8	xrw
default	0x0000000000000000	0xffffffffffffffff	

LINKER MAP FILES

- ❑ **Linker script and memory map section** contains a breakdown of the memory contribution of each and every file that was linked into the final image.

Linker script and memory map

```
.text          0x00000000000001000      0x8c8
**(.isr_vector)**
.isr_vector    0x00000000000001000      0x200 _build/nrf52840_xxaa/**gcc_startup_nrf52840.
              0x00000000000001000          __isr_vector

*(.text*)
.text          0x00000000000001200      0x40  /usr/local/Cellar/arm-none-eabi-gcc/8-2018-q
.text          0x00000000000001240      0x74  /usr/local/Cellar/arm-none-eabi-gcc/8-2018-q
              0x00000000000001240          _mainCRTStartup
              0x00000000000001240          _start
.text          0x000000000000012b4      0x3c  _build/nrf52840_xxaa/gcc_startup_nrf52840.S.
              0x000000000000012b4          Reset_Handler
              0x000000000000012dc          NMI_Handler

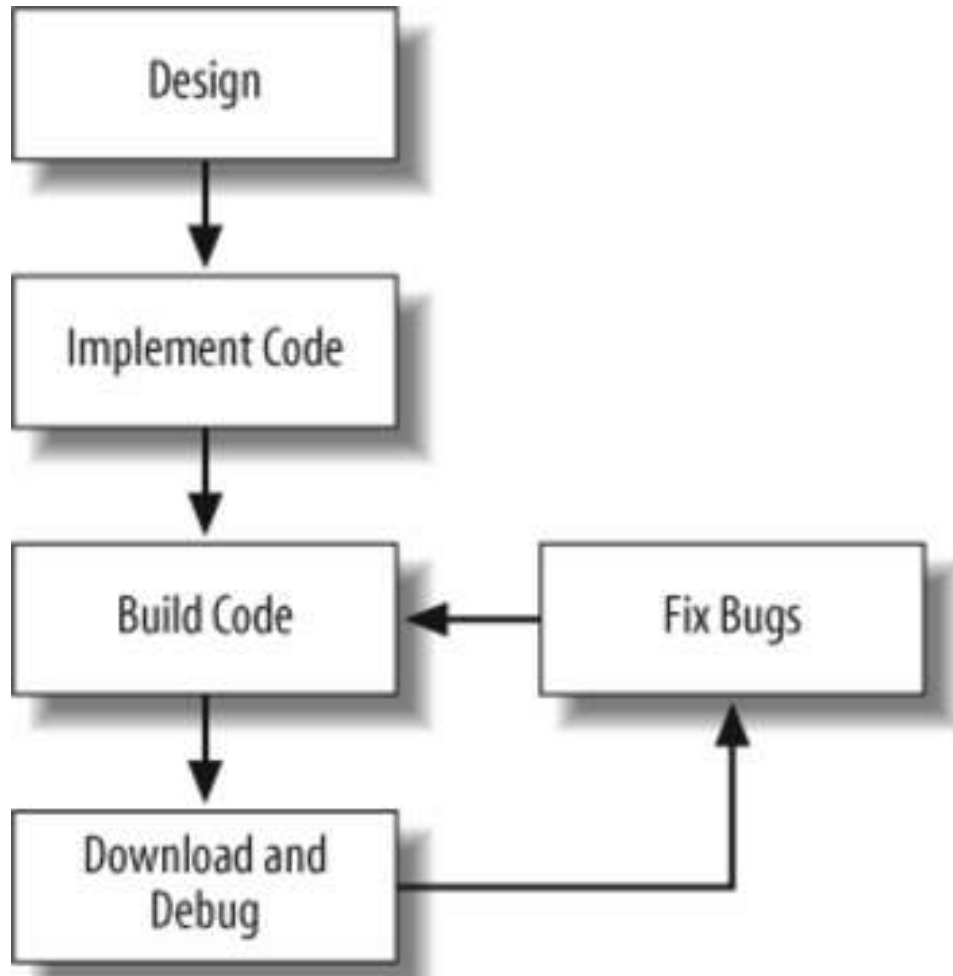
[...]

.text.**bsp_board_led_invert**
              0x000000000000012f0      **0x34** _build/nrf52840_xxaa/**boards.c.o**
              0x000000000000012f0          bsp_board_led_invert
```

LOADING ON THE TARGET

- ❑ Previously, we saw how the code or software to be executed on the embedded system (target board) is written on a computer.
- ❑ The resulting code created after subjecting it to be build process is called the *binary executable image or simply hex code*.
- ❑ This topic explains how the *hex code* is loaded on the target board which is referred as *downloading*
- ❑ And what are the various possible ways of *debugging a code* meant to run on a embedded system.

LOADING ON THE TARGET



❑ There are two ways of downloading the binary image on the embedded system:

1. Using a Device Programmer
2. In-system programming (ISP)

LOADING ON THE TARGET

1. Using a Device Programmer:

- Step 1: Once the binary image is ready on the computer, the device programmer is connected to the computer.
- Step 2: The muP/muC or memory chip, usually the ROM which is supposed to contain the binary image is placed on the proper socket on the device programmer.
- Step 3: The device programmer contains a software interface through which the user selects the target muP/muC for which the binary image has to be downloaded.
- Step 4: The Device programmer then transfers the binary image bit by bit to the chip.

LOADING ON THE TARGET

1. Using a Device Programmer:



Host Computer



muP/muC/Memory Chip

Device Programmer

LOADING ON THE TARGET

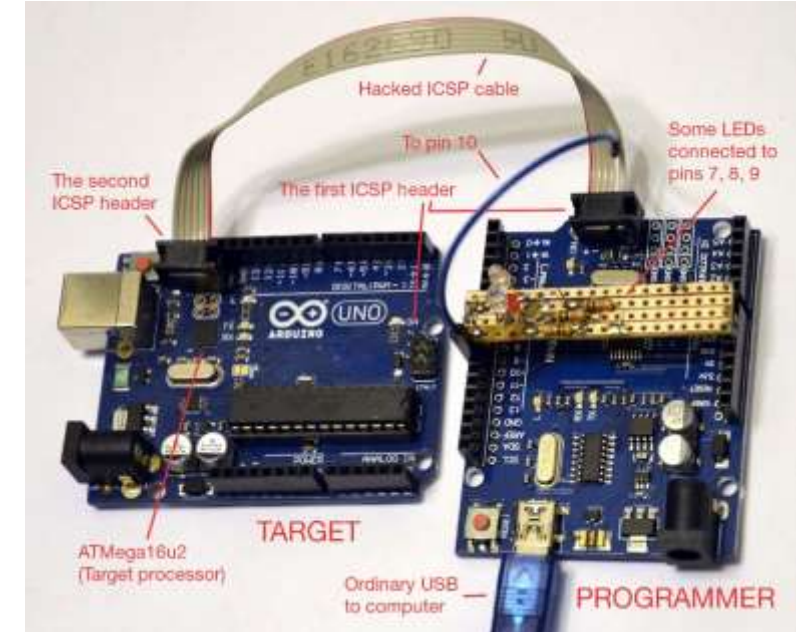
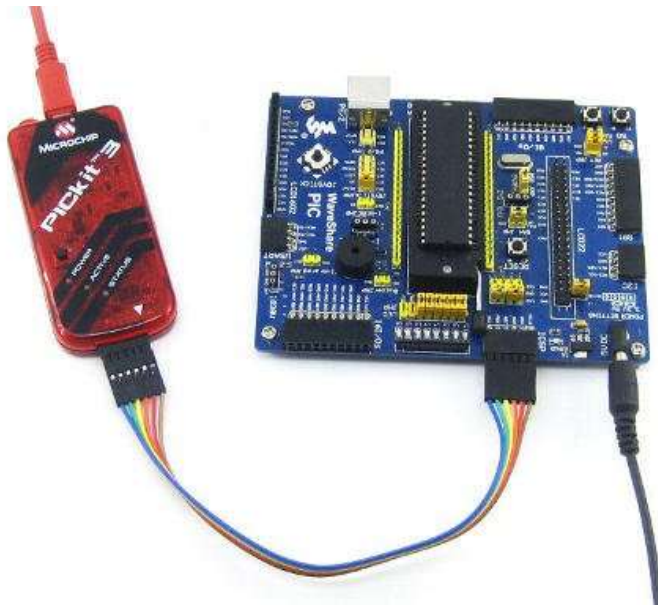
2. Using In System Programmer(ISP):

- Certain Target embedded platforms uses a piece of hardware called ISP that have a hardware interface to both the computer as well the target board's chip where the code is to be downloaded.
- ISP is also called in-circuit serial programming (ICSP), is the ability of programming the embedded processor/controller while it is installed in a complete system, rather than requiring the chip to be programmed prior to installing it into the system.
- It also allows firmware updates to be delivered to the on-chip memory of microcontrollers and related processors without requiring specialist programming circuitry on the circuit board, and simplifies design work

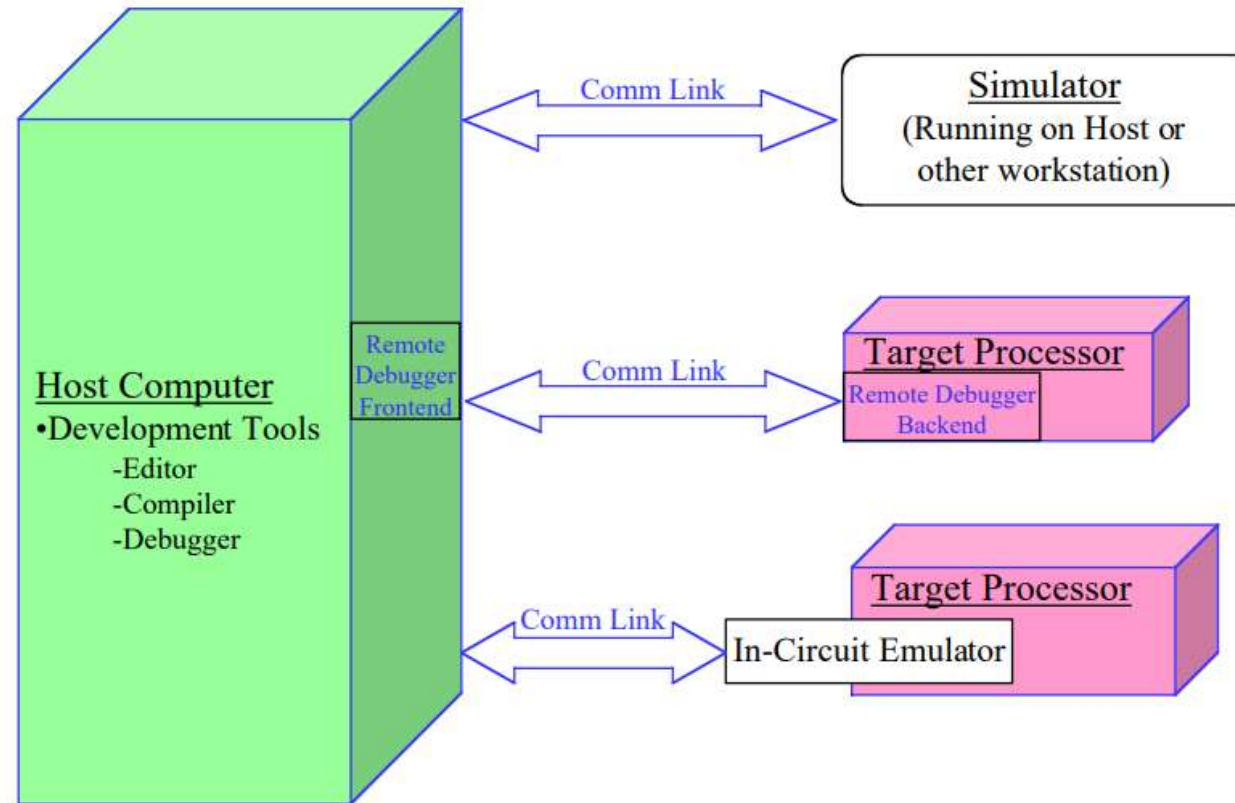
LOADING ON THE TARGET

2. Using In System Programmer(ISP):

- The user through the ISP's software interface sends the binary image to the target board. This avoids the requirement of frequently removing the microprocessor / microcontroller or ROM for downloading the code if a device programmer had to be used.



DEBUGGING TOOLS



DEBUGGING TOOLS

SIMULATOR

- ❑ Simulator is a host-based program that simulates functionality and instruction set of target processor.
- ❑ The front-end has text or GUI-based windows for source code, register contents, etc.
- ❑ Simulators are valuable during early stages of development.
- ❑ Disadvantage of this method is that, it only simulates the processor, and not the peripherals.

DEBUGGING TOOLS

REMOTE DEBUGGER

- ❑ Remote debuggers are one of the commonly used downloading and testing tools during development of embedded software
- ❑ It is used to monitor/control embedded SW. It is used to download, execute and debug embedded software over a comm. link.
- ❑ The program running on the host of a remote debugger has a user interface (GUI/Command-line) that looks just like other debugger
- ❑ The front-end of the GUI debuggers contain several windows to show the active part of the source code, current register contents, and other relevant information about the executing program.

DEBUGGING TOOLS

REMOTE DEBUGGER

- ❑ Backend provides low-level control of target processor, runs on target processor and communicates to the front-end over a communication link.
- ❑ Debugger and software being debugged are executing on two different computer systems.
 - Start/restart/kill, and stepping through program.
 - Software breakpoints.
 - Reading/writing registers or data at specified address.
- ❑ Disadvantage: inability to debug startup Code, code must execute from RAM, requires a target processor to run the final software package

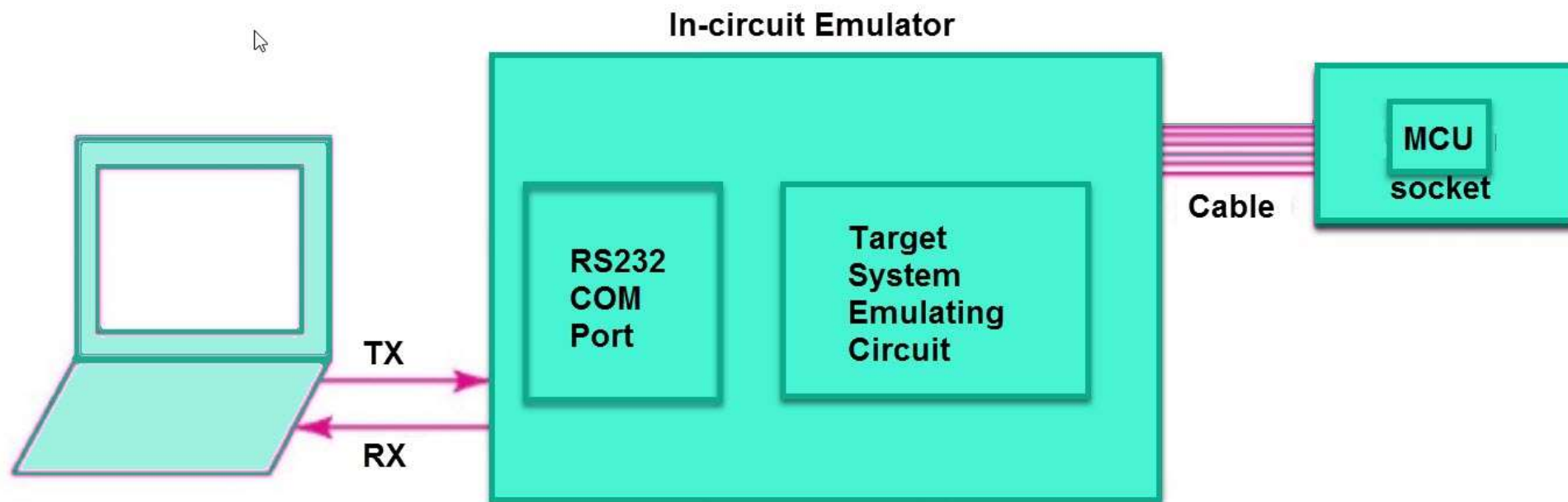
DEBUGGING TOOLS

IN-CIRCUIT EMULATOR (ICE)

- ❑ Emulation refers to the ability of a computer program or electronic device to imitate another program or device.
- ❑ An emulator is a piece of hardware/software that enables one computer system to run programs that are written for another computer system.
- ❑ An in-circuit emulator (ICE) provides a lot more functionality than a remote debugger.
- ❑ In addition to providing the features available with a remote debugger, an ICE allows you to debug startup code and programs running from ROM, etc.,

DEBUGGING TOOLS

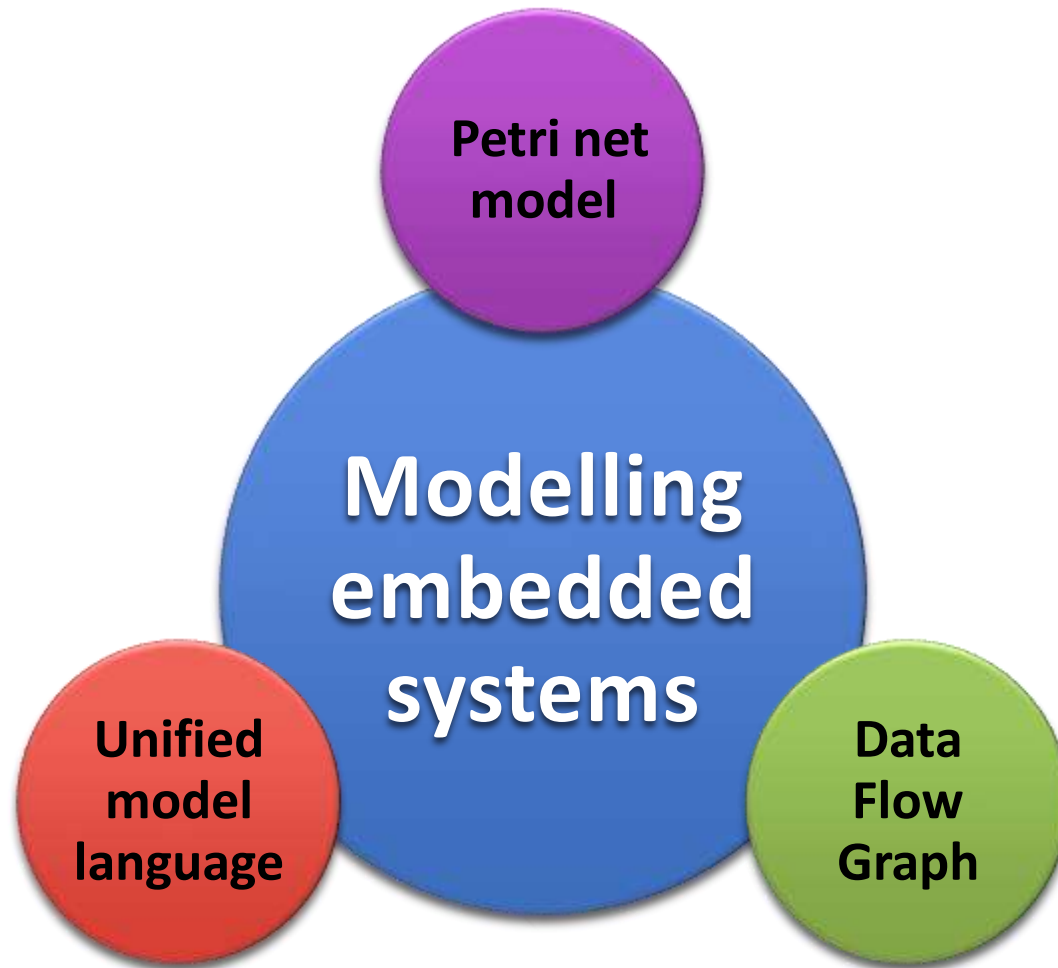
IN-CIRCUIT EMULATOR (ICE)



DEBUGGING TOOLS

IN-CIRCUIT EMULATOR (ICE)

- ❑ In-Circuit Emulator (ICE) takes the place of the target processor. It contains a copy of target processor, plus RAM, ROM, and its own embedded software.
- ❑ It allows you to examine the state of the processor while the program is running. It uses the remote debugger for human interface.
- ❑ ICE provides greater flexibility, ease for developing various applications on a single system in place of testing that multiple targeted systems. Disadvantage of this method is that, it is expensive.



Modelling Programs

Representation of a program

- ❑ **Modelling the programs in Embedded systems.**
- ❑ **Program has basic blocks, conditional statements etc.**
- ❑ **Can be represented by**
 - ❑ **Data Flow graph (DFG)**
 - ❑ **Control Data Flow Graph (CDFG)**

Modelling Programs

Representation of a program

- ❑ **Model a program that has no conditionals**
- ❑ **Basic blocks - having one entry and exit point.**
- ❑ **Before go for DFG represent the program in single assignment form**
- ❑ **Single assignment form**
 - **Having statement in a program, where the assignment variables (in left) appears only one.**
 - **It verifies no cyclic form in the statements or blocks.**
 - **It identifies the unique location of variables in the code**
 - **Any use of repeated assignment to single variable need to be rewrite and assigns the latest assigned variable if it is used further.**

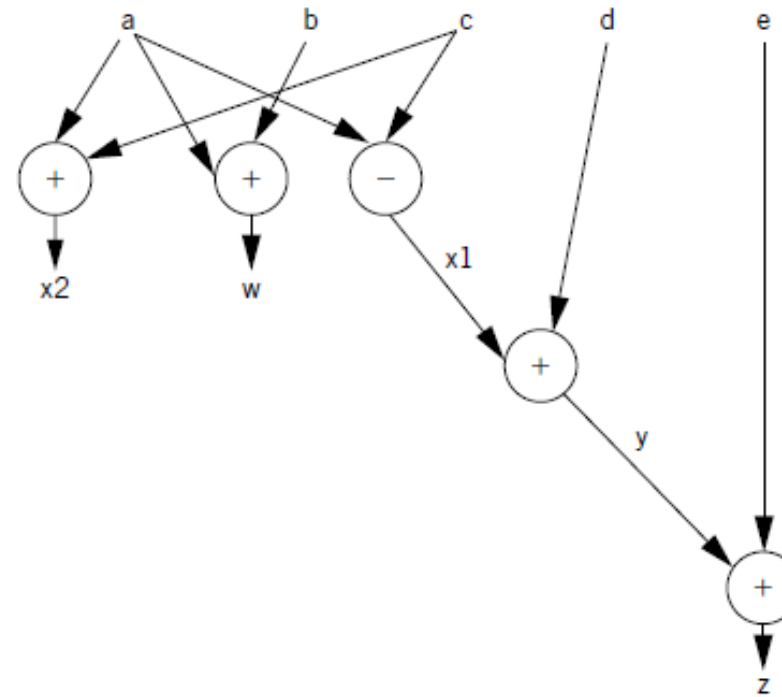
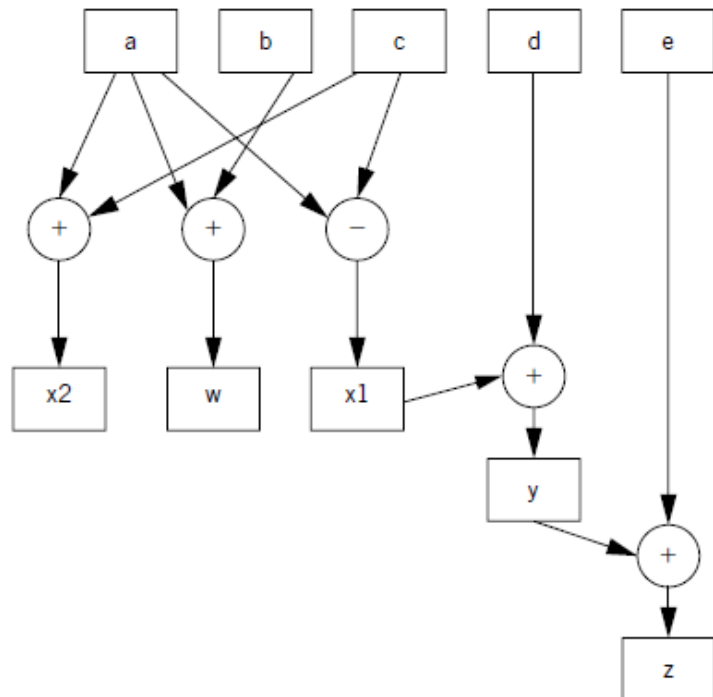
Modelling Programs

Representation of a program

- ❑ Two types of nodes
- ❑ Round nodes - represents or denotes operators.
- ❑ Square nodes - represents values.

Modelling Programs

Single assignment form



Modelling Programs

Advantages of DFG

- ❑ It orders the way operations can be performed in a program.
- ❑ Determines the feasible reordering's of the operations, this will reduce the pipeline or cache conflicts

Modelling Programs

Control DFG

- ❑ Constructs a model for decision flow and data flow of a program.
- ❑ Data flow nodes represented by basic block (set of statements).
- ❑ Control flow of a sequential program is represented by decision nodes

Modelling Programs

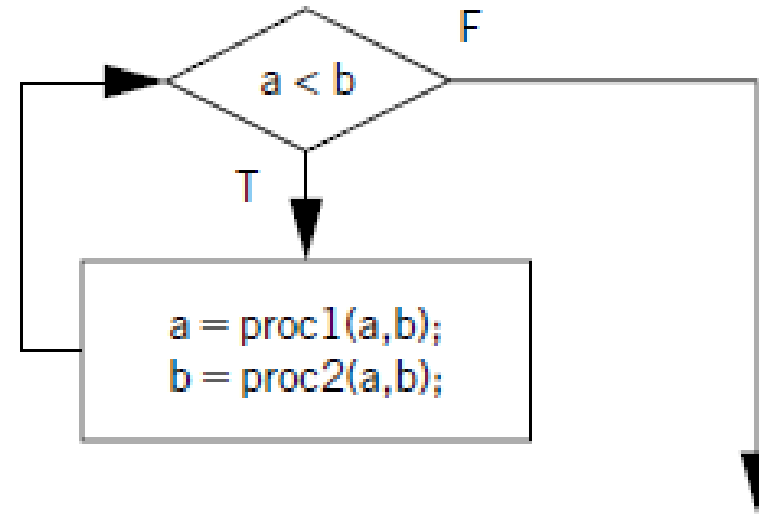
Control DFG

- ❑ **Basic block nodes**
- ❑ **Decision nodes**
- ❑ **Edges labelled with possible outcomes**

Modelling Programs

Building loop in CDFG

```
while (a < b)
{
a = proc1(a,b);
b= proc2(a,b);
}
```

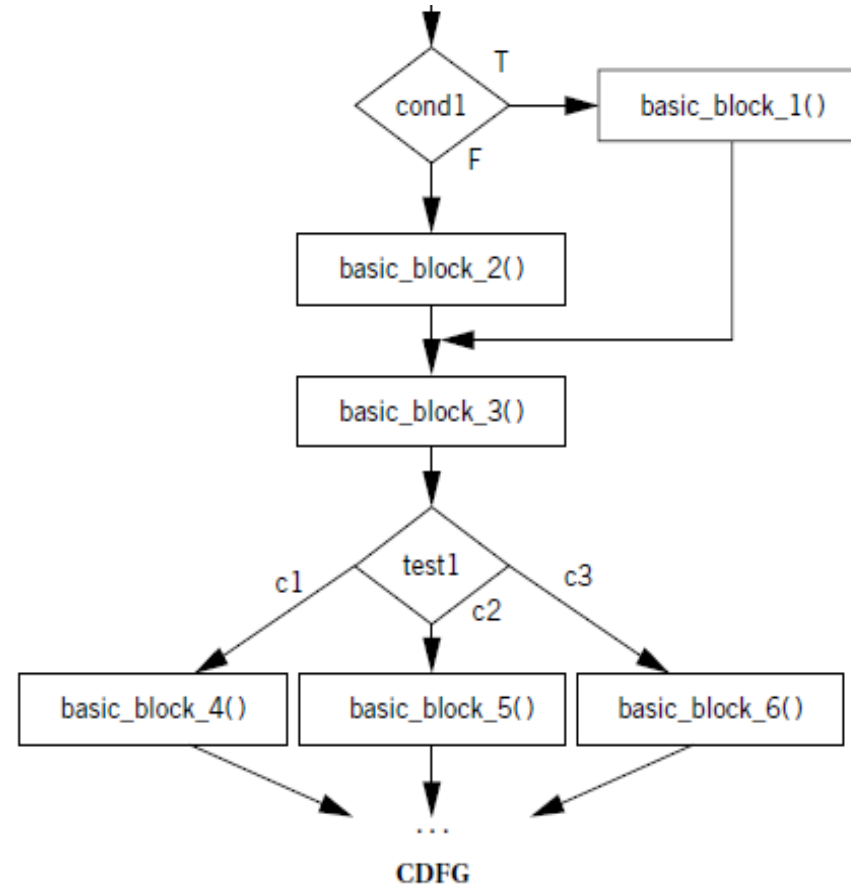


CDFG

Modelling Programs

Condition blocks using CDFG

```
if (cond1)
    basic_block_1( );
else
    basic_block_2();
basic_block_3( );
switch (test1) {
case c1: basic_block_4( ); break;
case c2: basic_block_5( ); break;
case c3: basic_block_6( ): break;
}
```



Modelling Programs

Control DFG

- ❑ CDFG is a hierarchical representation.
- ❑ Each block in CDFG is expanded using Data flow graph.

PETRINET MODEL

PETRI NET MODEL

- ❑ Petri net can also called as Place/Transition net
- ❑ It is used for describing and analyzing concurrent process
- ❑ Petri net is also a graphical tool
- ❑ Petri Net(PN) is Very similar to State Transition Diagrams which model the system behavior
- ❑ It is an abstract model to show the interaction between asynchronous processes
- ❑ In asynchronous process, start and sequence of processes may vary during the execution

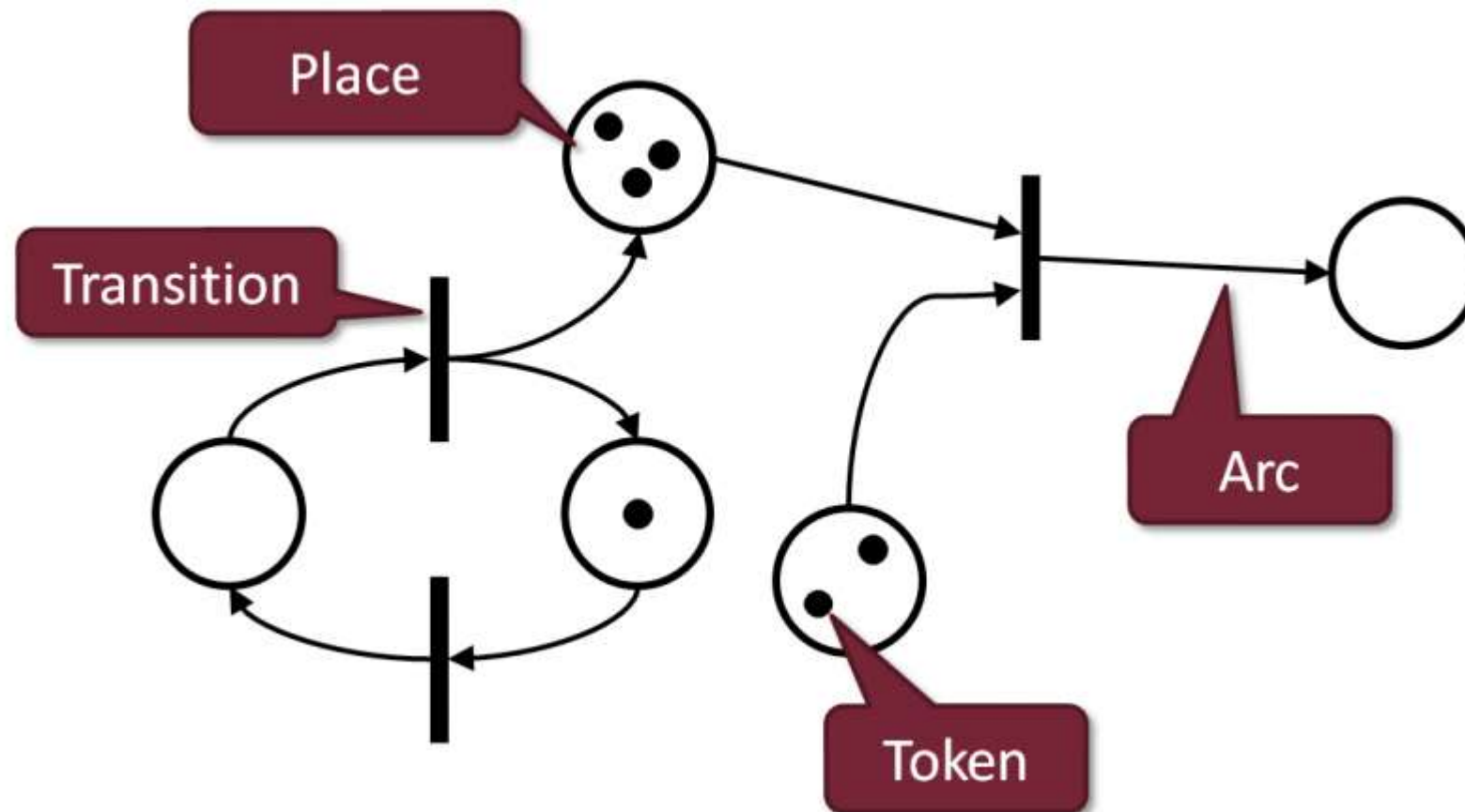
PETRINET MODEL

COMPONENTS OF PETRINET MODEL

- ❑ **Petri net Model consists of four types of components - places (circles), transitions (rectangles) and arcs (arrows), tokens(black dot):**
 - ❑ **Places represent possible states of the system;**
 - ❑ **Transitions are events which cause the change of state**
 - ❑ **Every arc simply connects a place with a transition or a transition with a place.**
 - ❑ **Change of state is denoted by a movement of token(s) from place(s) to place(s)**

PETRINET MODEL

COMPONENTS OF PETRINET MODEL



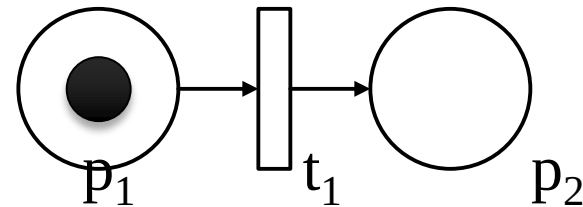
PETRINET MODEL

- ❑ Change of state is caused by the firing of a transition
- ❑ Firing refers to the occurrence of an event
- ❑ The firing is based on the input conditions, denoted by token availability
- ❑ A transition is enabled when sufficient tokens are available in input places
- ❑ Once firing is initiated, tokens will be transferred from input to output places

PETRINET MODEL

COMPONENTS OF PETRINET

- Below is an example Petri net with two places and one transaction.

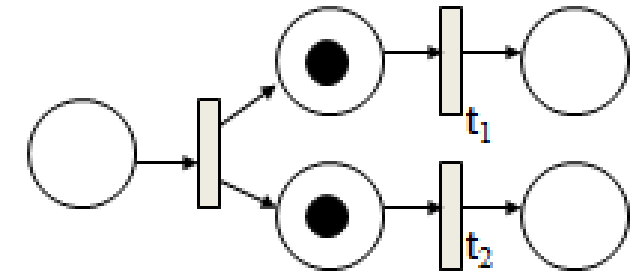
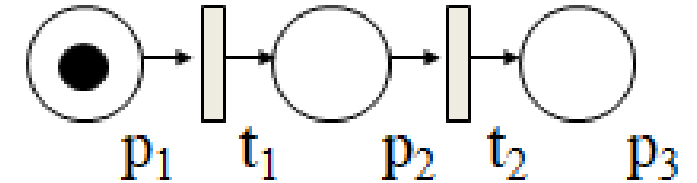


- p_1 : input place
- p_2 : output place

PETRINET MODEL

PROPERTIES

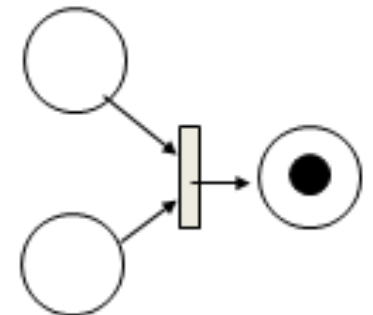
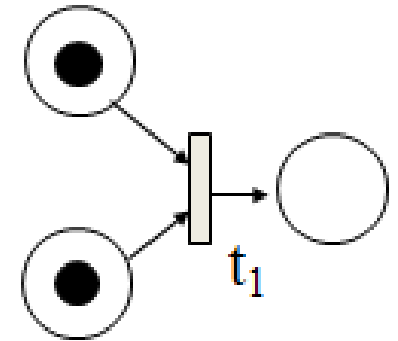
- ❑ **Sequential Execution:** Transition t_2 can take place only after t_1 . Here a constraint is imposed “ t_2 after t_1 ”
- ❑ **Concurrency:** This property is used in Modelling distributed control system. t_1 and t_2 are concurrent process



PETRINET MODEL

PROPERTIES

- ❑ **Merging:** When several tokens are arrived at same transition merging will take place
- ❑ **Synchronization:** Transition t_1 will be enabled only when at least one token is available at each of its input places



PETRINET MODEL

EXAMPLE-1 POINT OF SALE (POS) MACHINE

- ❑ A point of sale terminal (POS terminal) is an electronic device used to process card payments at retail locations. A POS terminal generally does the following:
 - Reads the information off a customer's credit or debit card
 - Validate the user using 4-digit PIN number
 - Checks whether the funds in a customer's bank account are sufficient
 - Transfers the funds from the customer's account to the seller's account
 - Records the transaction and prints a receipt

PETRINET MODEL

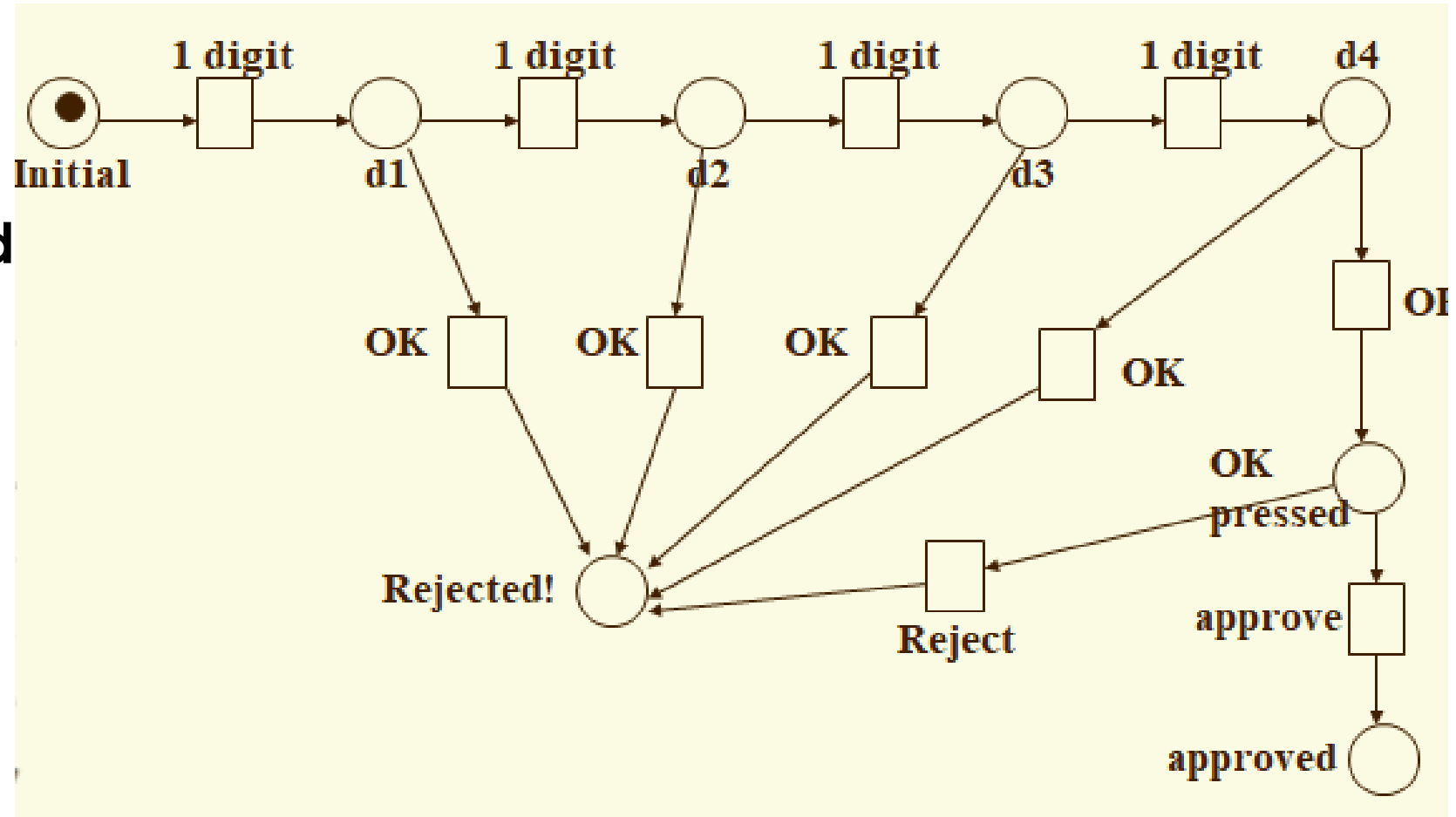
EXAMPLE-1 POINT OF SALE (POS) MACHINE

Scenario 1: Normal

- Enters all 4 digits and press OK.

Scenario 2: Exceptional

- Enters only 3 digits and press OK.



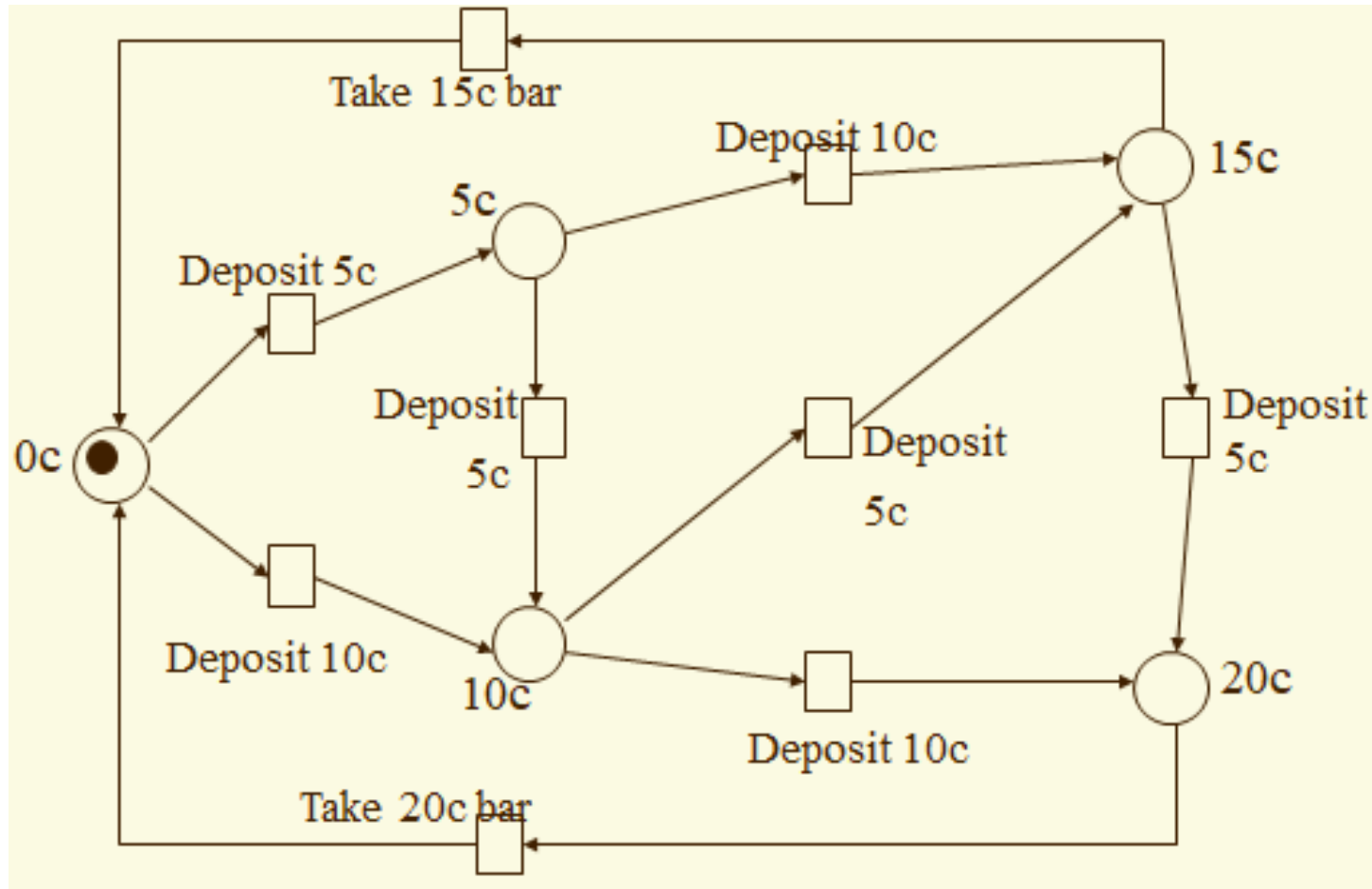
PETRINET MODEL

EXAMPLE-2 VENDING MACHINE

- ❑ The machine dispenses two kinds of snack bars 20c and 15c.
Constraint: 10c and 5c coins can only be used
- ❑ Scenario 1:
 - Deposit four 5c, take 20c snack bar.
- ❑ Scenario 2:
 - Deposit 10 + 5c, take 15c snack bar.
- ❑ Scenario 3:
 - Deposit 5 + 10 + 5c, take 20c snack bar.

PETRINET MODEL

EXAMPLE-2 VENDING MACHINE



PETRINET MODEL

EXAMPLE-3 ORDER MANAGEMENT IN A RESTAURANT

- ❑ In the real world, many events are concurrent in nature. Many applications have global state formed from local state.
- ❑ Scenario 1:
 - ❑ Waiter takes order from customer 1
 - ❑ Serves customer 1
 - ❑ Takes order from customer 2
 - ❑ Serves customer 2.
- ❑ Scenario 2:
 - ❑ Waiter takes order from customer 1
 - ❑ Takes order from customer 2
 - ❑ Serves customer 2
 - ❑ Serves customer 1

PETRINET MODEL

EXAMPLE-3 ORDER MANAGEMENT IN A RESTAURANT

